

PostgreSQL Migration Runbook

Standalone VM (On-Premise / Private Cloud) ↔ AWS

Comprehensive Guide for DDL & Data Migration

Version 2.0 | April 2026

Classification: CONFIDENTIAL — Internal Use Only

Covers: PostgreSQL 13 / 14 / 16 / 17

AWS Targets: RDS PostgreSQL | Aurora PostgreSQL | EC2 Self-Managed

Tools: pg_dump / pg_restore | pg_basebackup | pgBackRest | AWS DMS | AWS SCT

Table of Contents

- 1. Document Purpose & Scope**
- 2. Migration Architecture Overview**
- 3. Section 1 — Pre-Migration Planning & Prerequisites**
 - 3.1 Infrastructure & Network Requirements
 - 3.2 Source Database Assessment
 - 3.3 AWS Target Environment Setup
 - 3.4 Security & Compliance Checklist
- 4. Section 2 — Migration Methods Comparison**
 - 4.1 Method 1: pg_dump / pg_restore (Logical Migration)
 - 4.2 Method 2: pg_basebackup (Physical / Binary Migration)
 - 4.3 Method 3: pgBackRest (Backup-Based Migration)
 - 4.4 Method 4: AWS Database Migration Service (DMS)
 - 4.5 Method 5: AWS Schema Conversion Tool (SCT) + DMS
 - 4.6 Method 6: Logical Replication (Native)
 - 4.7 Method Selection Matrix
- 5. Section 3 — Migrating DDL Separately**
- 6. Section 4 — Migrating Data Separately**
- 7. Section 5 — Combined DDL + Data Migration (Recommended Procedure)**
- 8. Section 6 — Reverse Migration: AWS → Standalone VM**
- 9. Section 7 — Post-Migration Validation & Checks**
- 10. Section 8 — Enterprise Best Practices & MNC Patterns**
- 11. Section 9 — Rollback Procedures**
- 12. Section 10 — Troubleshooting Common Issues**
- 13. Appendix A — Command Quick Reference**
- 14. Appendix B — Migration Checklist (Printable)**

1. Document Purpose & Scope

This runbook provides a comprehensive, step-by-step guide for migrating PostgreSQL databases between standalone virtual machines (on-premise data centers or private cloud environments) and Amazon Web Services (AWS). It covers both directions — VM-to-AWS and AWS-to-VM — and addresses DDL-only migration, data-only migration, and combined migration scenarios.

Scope Includes:

- PostgreSQL versions 13, 14, 16, and 17
- Source: AlmaLinux 8/9 VMs (on-premise or private cloud)
- Target AWS services: Amazon RDS for PostgreSQL, Amazon Aurora PostgreSQL-Compatible, EC2 self-managed PostgreSQL
- All three backup tools equally: pg_dump/pg_restore, pg_basebackup, and pgBackRest
- AWS-native migration services: DMS and SCT
- Enterprise patterns used by top MNC companies (FAANG-scale, Fortune 500 financial institutions, SaaS providers)

Environment Path Standards

Throughout this runbook, all paths follow the established environment standard:

Data Directory: /apps/pgsql_data/<PG_VERSION>

WAL Archive: /apps/pgsql_archives/

Logs: /apps/logs

Backup Images: /apps/backups/

2. Migration Architecture Overview

A PostgreSQL migration from a standalone VM to AWS involves multiple layers — network connectivity, schema compatibility, data transfer, and validation. The architecture depends on the chosen AWS target and whether the migration is offline (with downtime) or online (near-zero downtime).

Migration Pattern	Downtime	Complexity	Best For
Offline (pg_dump/pg_restore)	Hours (size-dependent)	Low	Small-to-medium DBs (< 500 GB)
Offline (pg_basebackup)	Hours	Low-Medium	EC2 targets (same PG major version)
Offline (pgBackRest restore)	Hours	Medium	Large DBs with existing pgBackRest setup
Online (AWS DMS — Full + CDC)	Minutes	Medium-High	RDS/Aurora targets, heterogeneous migrations
Online (Native Logical Replication)	Minutes	Medium	Same or close PG versions, selective tables
Hybrid (SCT for DDL + DMS for Data)	Minutes	High	Complex schemas, cross-engine if needed

Industry Standard: MNC Migration Pattern

Top-tier companies (Google Cloud migration teams, AWS Professional Services, Accenture, Deloitte, Infosys) follow a standardized migration framework: **Assess → Mobilize → Migrate → Optimize**. This is aligned with AWS's Migration Acceleration Program (MAP). The procedure in this runbook mirrors this framework.

3. Section 1 — Pre-Migration Planning & Prerequisites

Proper pre-migration planning is the single most important factor in a successful migration. Every major migration failure at enterprise scale traces back to insufficient assessment or skipped prerequisites.

3.1 Infrastructure & Network Requirements

Network Connectivity Options (Source VM → AWS):

Connectivity Method	Bandwidth	Latency	Security	Cost	Use Case
AWS Direct Connect	1–100 Gbps	Low (< 5ms)	Private, encrypted	\$\$\$\$	Large DBs, production workloads
Site-to-Site VPN	Up to 1.25 Gbps	Medium	IPSec encrypted	\$\$	Medium DBs, existing VPN infra
Internet (TLS/SSH)	Variable	Variable	TLS + SSH tunnel	\$	Small DBs, dev/test migrations
AWS Snowball Edge	N/A (physical)	N/A	Encrypted device	\$\$\$	Multi-TB offline data transfer
AWS Transfer Family	Variable	Variable	SFTP/FTPS	\$\$	File-based transfer (dump files)

CRITICAL — Bandwidth Estimation

Calculate required bandwidth: $\text{DB_Size_GB} / (\text{Available_Window_Hours} \times 3600) \times 8 = \text{Required_Mbps}$. For a 500 GB database with a 4-hour maintenance window: $500 / (4 \times 3600) \times 8 \approx 278$ Mbps minimum. Always add 30% overhead for protocol and retransmission overhead. Top MNCs typically provision 2× the calculated bandwidth to handle unexpected spikes.

Firewall / Security Group Rules Required:

- Source VM → AWS RDS/Aurora endpoint: TCP port 5432 (or custom PG port)
- Source VM → AWS DMS replication instance: TCP port 5432
- SSH tunnel (if used): TCP port 22 from bastion/jump host
- pgBackRest repository access (if S3-based): HTTPS 443 to S3 endpoints
- NTP synchronization: UDP 123 (clock skew causes replication issues)

3.2 Source Database Assessment

A thorough assessment of the source database is mandatory before any migration work begins. This assessment forms the basis for choosing the right migration method and estimating timelines.

Step 1: Collect Database Inventory

```
-- Run on source PostgreSQL instance
-- 1. Database size inventory
SELECT datname, pg_size_pretty(pg_database_size(datname)) AS size
FROM pg_database WHERE datistemplate = false ORDER BY pg_database_size(datname) DESC;

-- 2. Table count and largest tables per database
SELECT schemaname, tablename, pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS total_size,
       pg_size_pretty(pg_relation_size(schemaname||'.'||tablename)) AS table_size,
       pg_size_pretty(pg_indexes_size(schemaname||'.'||tablename)) AS index_size
FROM pg_tables WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC LIMIT 20;

-- 3. PostgreSQL version
SELECT version();

-- 4. Extension inventory (CRITICAL for RDS compatibility)
SELECT extname, extversion FROM pg_extension ORDER BY extname;

-- 5. Custom data types, domains, enums
SELECT typename, typtype, typnamespace::regnamespace FROM pg_type
WHERE typnamespace NOT IN ('pg_catalog'::regnamespace, 'information_schema'::regnamespace)
AND typtype IN ('c', 'd', 'e');
```

Step 2: Identify Migration Blockers for RDS/Aurora

```
-- Extensions NOT supported on RDS (check current list in AWS docs)
-- Common blockers: pg_cron (supported now), pgaudit (supported), adminpack (NOT on RDS)
SELECT extname FROM pg_extension
WHERE extname NOT IN (
    'plpgsql', 'pg_stat_statements', 'pgcrypto', 'uuid-oss', 'hstore', 'citext',
    'btree_gin', 'btree_gist', 'pg_trgm', 'postgis', 'postgis_topology',
    'fuzzystrmatch', 'tablefunc', 'intarray', 'pg_buffercache', 'pgaudit',
    'pg_cron', 'pglogical', 'pg_partman', 'pg_hint_plan', 'aws_s3', 'aws_lambda'
);

-- Check for tablespaces usage (RDS does not support custom tablespaces)
SELECT spcname, pg_size_pretty(pg_tablespace_size(spcname))
FROM pg_tablespace WHERE spcname NOT IN ('pg_default', 'pg_global');

-- Check for roles with superuser (RDS uses rds_superuser instead)
SELECT rolname, rolsuper, rolreplication FROM pg_roles WHERE rolsuper = true;

-- Check for event triggers (may need adjustment)
SELECT evtname, evtevent, evtowner::regrole FROM pg_event_trigger;
```

RDS/Aurora Compatibility Blockers

The following are **hard blockers** that must be resolved before migration to RDS/Aurora:

- Custom tablespaces → Must remap to `pg_default`
- Unsupported extensions → Must find alternatives or remove
- Superuser-dependent logic → Convert to `rds_superuser` grants
- Direct filesystem access (`COPY FROM local file`) → Use `S3 + aws_s3` extension
- `pg_hba.conf` customizations → Map to RDS Parameter Groups + Security Groups
- Custom `shared_preload_libraries` → Verify RDS support

Step 3: Assess Schema Complexity

```
-- Count of each object type
SELECT 'Tables' AS object_type, count(*) FROM pg_tables WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
UNION ALL
SELECT 'Indexes', count(*) FROM pg_indexes WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
UNION ALL
SELECT 'Views', count(*) FROM pg_views WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
UNION ALL
SELECT 'Mat. Views', count(*) FROM pg_matviews
UNION ALL
SELECT 'Functions', count(*) FROM pg_proc p JOIN pg_namespace n ON p.pronamespace=n.oid
  WHERE n.nspname NOT IN ('pg_catalog', 'information_schema')
UNION ALL
SELECT 'Triggers', count(*) FROM pg_trigger WHERE NOT tgisinternal
UNION ALL
SELECT 'Sequences', count(*) FROM pg_sequences WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
UNION ALL
SELECT 'Foreign Tables', count(*) FROM information_schema.foreign_tables;
```

Step 4: Record Replication & Connection Configuration

```
-- Current replication slots (if any – must be cleaned before migration)
SELECT slot_name, slot_type, active, xmin FROM pg_replication_slots;

-- Current replication configuration
SHOW wal_level;           -- Must be 'logical' for logical replication / DMS
SHOW max_replication_slots; -- Need slots for DMS or logical rep
SHOW max_wal_senders;     -- Must accommodate migration connections
SHOW max_connections;     -- Record for target sizing

-- Active connections baseline
SELECT count(*), state FROM pg_stat_activity GROUP BY state;
```

3.3 AWS Target Environment Setup

Choose your AWS target based on requirements:

Criteria	RDS PostgreSQL	Aurora PostgreSQL	EC2 Self-Managed
Management overhead	Fully managed	Fully managed	Self-managed (full control)
Max storage	64 TB (gp3/io2)	128 TB (auto-grows)	Instance-dependent (EBS)
Read replicas	Up to 15	Up to 15 (shared storage)	Manual setup
Failover time	1–2 minutes	< 30 seconds	Manual or custom HA
Point-in-time recovery	Yes (35-day retention)	Yes (35-day retention)	Manual (pgBackRest/pg_basebackup)
Custom extensions	AWS-approved only	AWS-approved only	Any extension
Custom tablespaces	No	No	Yes
pg_hba.conf control	No (use Security Groups)	No (use Security Groups)	Full control
OS-level access	No	No	Full SSH access
Cost model	Instance + storage	Instance + I/O + storage	Instance + EBS + self-manage
Best for	Standard workloads	High performance, HA	Max flexibility, custom needs

RDS/Aurora — Pre-Provisioning Checklist:

- Create a custom DB Parameter Group matching source postgresql.conf settings
- Create a custom DB Cluster Parameter Group (Aurora only)
- Configure VPC, Subnets (multi-AZ), and DB Subnet Group
- Create Security Groups allowing inbound 5432 from source VM IP/VPN CIDR
- Create IAM roles for S3 access (if using aws_s3 extension for COPY operations)
- Enable Enhanced Monitoring and Performance Insights
- Configure encryption at rest (KMS key) and in transit (SSL enforcement: rds.force_ssl=1)
- Set up CloudWatch alarms: FreeableMemory, CPUUtilization, ReadLatency, WriteLatency, DiskQueueDepth

EC2 Self-Managed — Pre-Provisioning Checklist:

- Launch EC2 instance with AlmaLinux 8/9 AMI
- Attach and mount EBS volumes with standard paths:
 - /apps/pgsql_data/<PG_VERSION> — gp3 or io2 (data)
 - /apps/pgsql_archives/ — gp3 (WAL archive)
 - /apps/backups/ — gp3 or S3-backed (backups)
 - /apps/logs — gp3 (logs)
- Install PostgreSQL from PGDG repository matching source version
- Configure pg_hba.conf, postgresql.conf to match source (adjusted for AWS networking)
- Install and configure pgBackRest with S3 repository backend

3.4 Security & Compliance Checklist

Enterprise Security Requirement

All migration traffic **MUST** be encrypted in transit. This is a non-negotiable requirement at every major enterprise (HIPAA, PCI-DSS, SOC 2, GDPR all mandate encryption of data in motion). Use SSL/TLS for all PostgreSQL connections and SSH tunnels or VPN for all data transfer channels.

- Verify source database SSL configuration: **SHOW ssl;** — must return 'on'
- Generate or import SSL certificates for target
- Create dedicated migration user with minimum required privileges:

```
-- On source: create migration user
CREATE USER migration_user WITH PASSWORD 'StrongP@ss!' REPLICATION;
GRANT CONNECT ON DATABASE mydb TO migration_user;
GRANT USAGE ON SCHEMA public TO migration_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO migration_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO migration_user;

-- For pg_dump (needs pg_read_all_data in PG 14+)
GRANT pg_read_all_data TO migration_user; -- PG 14+

-- For logical replication / DMS
ALTER USER migration_user WITH REPLICATION;
```

- Document and rotate all migration credentials post-migration
- Enable audit logging on both source and target during migration window
- Ensure backup of source is taken **IMMEDIATELY** before migration begins

4. Section 2 — Migration Methods Comparison

This section details every practical method for migrating PostgreSQL from a standalone VM to AWS. Each method is presented with exact commands, advantages, limitations, and recommended scenarios.

4.1 Method 1: `pg_dump` / `pg_restore` (Logical Migration)

The most universally applicable method. Works across PostgreSQL major versions, between any source/target combination, and is the primary method recommended by AWS documentation for migrations under 500 GB.

Advantages:

- Cross-version compatibility (can migrate PG 13 → PG 17)
- Works with all AWS targets (RDS, Aurora, EC2)
- Can separate DDL and data migration
- Supports parallel jobs for faster restoration
- Selective migration (specific schemas, tables)

Limitations:

- Requires downtime (offline migration)
- Duration proportional to database size
- Does not capture ongoing changes during dump
- Large databases (> 500 GB) can take many hours

Full Migration Procedure:

```
# ████████████████████████████████████████████████████████████████████████████  
# STEP 1: Pre-flight checks on source  
# ████████████████████████████████████████████████████████████████████████████  
# Verify available disk space for dump file  
df -h /apps/backups/  
  
# Estimate dump size (compressed custom format is ~30-40% of DB size)  
psql -c "SELECT pg_size_pretty(pg_database_size('mydb'));"  
  
# ████████████████████████████████████████████████████████████████████████████  
# STEP 2: Take consistent dump (custom format for parallel restore)  
# ████████████████████████████████████████████████████████████████████████████  
pg_dump -h localhost -U postgres -d mydb \  
--format=custom \  
--compress=6 \  
--verbose \  
--jobs=4 \  
--file=/apps/backups/mydb_full_$(date +%Y%m%d_%H%M%S).dump \  
>&1 | tee /apps/logs/pg_dump_$(date +%Y%m%d).log  
  
# ████████████████████████████████████████████████████████████████████████████  
# STEP 3: Transfer dump file to AWS  
# ████████████████████████████████████████████████████████████████████████████  
# Option A: Direct SCP to EC2 bastion/target  
scp -C /apps/backups/mydb_full_20260408.dump \  
ec2-user@bastion.example.com:/apps/backups/  
  
# Option B: Upload to S3 (for RDS, use aws_s3 extension or restore from EC2)  
aws s3 cp /apps/backups/mydb_full_20260408.dump \  
s3://my-migration-bucket/pg-migration/ \  
--sse AES256  
  
# ████████████████████████████████████████████████████████████████████████████  
# STEP 4: Restore to AWS target  
# ████████████████████████████████████████████████████████████████████████████  
# For RDS/Aurora (connect from EC2 in same VPC):  
pg_restore -h mydb.xxxx.us-east-1.rds.amazonaws.com \  
-U masteruser -d mydb \  
--format=custom \  
--jobs=4 \  
--no-owner \  
--no-privileges \  
--verbose \  
/apps/backups/mydb_full_20260408.dump \  
>&1 | tee /apps/logs/pg_restore_$(date +%Y%m%d).log  
  
# For EC2 self-managed:  
pg_restore -h localhost -U postgres -d mydb \  
--format=custom \  
--jobs=4 \  
--verbose \  
/apps/backups/mydb_full_20260408.dump \  
>&1 | tee /apps/logs/pg_restore_$(date +%Y%m%d).log
```

Performance Tip — Parallel Restore

Use `--jobs=N` where N = number of CPU cores on the target (typically 4–8 for migration). This parallelizes the restore of tables and indexes. For RDS/Aurora, run `pg_restore` from an EC2 instance in the **same VPC and AZ** to minimize network latency. This alone can cut restore time by 50–70%.

pgBackRest S3 Performance

pgBackRest with S3 and zstd compression can achieve backup throughput of 200–500 MB/s depending on instance size and network. Use **process-max=4** (or higher) to parallelize. For databases over 1 TB, pgBackRest with S3 is often faster than pg_dump due to parallelized file-level backup.

4.4 Method 4: AWS Database Migration Service (DMS)

AWS DMS enables near-zero-downtime migrations by performing an initial full load followed by ongoing Change Data Capture (CDC). This is the AWS-recommended approach for production migrations where downtime must be minimized. Works with all AWS PostgreSQL targets (RDS, Aurora, EC2).

Prerequisites for DMS with PostgreSQL Source:

```
-- On source PostgreSQL:
-- 1. Set wal_level to logical (required for CDC)
ALTER SYSTEM SET wal_level = 'logical';
-- REQUIRES RESTART

-- 2. Increase max_replication_slots
ALTER SYSTEM SET max_replication_slots = 10;
-- REQUIRES RESTART

-- 3. Increase max_wal_senders
ALTER SYSTEM SET max_wal_senders = 10;
-- REQUIRES RESTART

-- 4. Set wal_sender_timeout
ALTER SYSTEM SET wal_sender_timeout = 0;

-- 5. Restart PostgreSQL
-- sudo systemctl restart postgresql-17

-- 6. Install test_decoding or pglogical (required for CDC)
-- test_decoding is built-in; pglogical needs separate install

-- 7. Grant replication privilege
ALTER USER migration_user WITH REPLICATION;
GRANT rds_replication TO migration_user; -- If source is also RDS
```

DMS Migration Steps:

- **Step 1:** Create DMS Replication Instance (same VPC as target, dms.r5.xlarge or larger)
- **Step 2:** Create Source Endpoint (PostgreSQL, connection details to source VM via VPN/Direct Connect)
- **Step 3:** Create Target Endpoint (PostgreSQL RDS/Aurora/EC2 endpoint)
- **Step 4:** Test both endpoint connections
- **Step 5:** Create Migration Task:
 - Migration type: **Migrate existing data and replicate ongoing changes**
 - Table preparation mode: **Do nothing** (if DDL migrated separately) or **Drop tables on target**
 - Enable validation: Yes
 - Enable CloudWatch logs: Yes
- **Step 6:** Start migration task and monitor via DMS console / CloudWatch
- **Step 7:** When CDC catches up (CDC latency → 0), perform cutover

DMS Limitations to Know

- DMS does **NOT** migrate: sequences (must sync manually), stored procedures/functions, triggers, views, materialized views, custom types, extensions, or roles/grants.
- DDL must be migrated separately (use `pg_dump --schema-only` or AWS SCT).
- Large objects (LOBs) require special LOB settings in the DMS task.
- Partitioned tables may need table-mapping rules.
- Always validate row counts and checksums post-migration.

4.5 Method 5: AWS Schema Conversion Tool (SCT) + DMS

AWS SCT analyzes your source schema, identifies incompatibilities with the target, generates a conversion report, and produces the DDL for the target. When combined with DMS for data migration, this provides a complete enterprise migration pipeline. This is the method used by AWS Professional Services for large engagements.

When to Use SCT:

- Large, complex schemas with hundreds of tables
- When you need an automated compatibility report
- Cross-version migrations where subtle DDL differences exist
- When migrating from other engines (Oracle → Aurora PostgreSQL) — but also useful PG → PG

SCT Workflow:

- 1. Download and install AWS SCT (desktop application)
- 2. Create project → Connect to source PostgreSQL
- 3. Run Assessment Report → Review conversion issues
- 4. Convert schema → Review generated DDL
- 5. Apply DDL to target AWS PostgreSQL
- 6. Use DMS for data migration (full load + CDC)

4.6 Method 6: Native Logical Replication

PostgreSQL's built-in logical replication can be used for near-zero-downtime migration. The source publishes changes, and the target subscribes. Once the target catches up, you cut over. This requires PG 10+ on both sides (PG 13+ recommended for robustness).

4.7 Method Selection Matrix — Choosing the Right Approach

Use this decision matrix to select the optimal migration method based on your specific requirements:

Scenario	Recommended Method	Rationale
DB < 100 GB, downtime OK	pg_dump / pg_restore	Simplest, most reliable, cross-version
DB < 500 GB, downtime OK	pg_dump / pg_restore (parallel)	Good speed with --jobs=4-8
DB > 500 GB, downtime OK, EC2 target	pgBackRest (S3 repo)	Fastest for large DBs, parallelized
DB > 500 GB, downtime OK, EC2 target, same PG version	pg_basebackup	Exact binary copy, fastest for same-version
Any size, minimal downtime, RDS/Aurora target	AWS DMS (Full + CDC)	AWS-native, near-zero downtime
Any size, minimal downtime, EC2 target	Native logical replication	No DMS cost, native PG feature
Complex schema, need compatibility report	AWS SCT + DMS	Automated analysis, enterprise-grade
Multi-TB, air-gapped source	pgBackRest → Snowball → S3 → EC2	Physical transfer for disconnected environments

Industry Best Practice — Recommended Combination

The approach used by most Fortune 500 companies and AWS Professional Services:

1. Use **pg_dump --schema-only** to migrate DDL (gives full control over schema review)
2. Use **AWS DMS** (Full Load + CDC) for data migration with near-zero downtime
3. Use **pg_dump --data-only** as a fallback/validation method
4. Sync sequences manually at cutover

This hybrid approach combines the reliability of logical DDL export with the minimal-downtime benefit of DMS.

5. Section 3 — Migrating DDL Separately

Separating DDL migration from data migration is the enterprise standard. It allows schema review, modification for target compatibility (e.g., removing tablespaces for RDS), and pre-creation of objects before data loading begins.

Step 1: Export DDL from Source

```
# Full schema export (all objects, no data)
pg_dump -h localhost -U postgres -d mydb \
  --schema-only \
  --no-owner \
  --no-privileges \
  --format=plain \
  --file=/apps/backups/mydb_ddl_$(date +%Y%m%d).sql

# Export schema for specific schema(s):
pg_dump -h localhost -U postgres -d mydb \
  --schema-only \
  --schema=public \
  --schema=app_schema \
  --format=plain \
  --file=/apps/backups/mydb_ddl_selected.sql

# Export only specific object types:
# Tables + indexes + constraints only (no functions/views/triggers):
pg_dump -h localhost -U postgres -d mydb \
  --schema-only \
  --no-owner --no-privileges \
  --section=pre-data \
  --format=plain \
  --file=/apps/backups/mydb_ddl_predata.sql
```

Step 2: Review and Modify DDL for Target Compatibility

This is a critical step. The exported DDL may contain constructs not supported on the target. Perform the following checks and modifications:

DDL Element	Source (VM)	Target (RDS/Aurora)	Action Required
TABLESPACE clauses	SET TABLESPACE custom_ts	Not supported	Remove all TABLESPACE clauses
OWNER TO statements	ALTER ... OWNER TO postgres	No superuser	Use --no-owner or change to rds master user
GRANT/REVOKE	GRANT ... TO custom_role	Role may not exist	Use --no-privileges; re-create grants manually
CREATE EXTENSION	CREATE EXTENSION adminpack	Not all supported	Remove unsupported; add supported ones
SET/ALTER SYSTEM	Various GUCs	Use Parameter Groups	Remove; configure via Parameter Group
COPY FROM local file	COPY FROM '/path/file.csv'	No filesystem access	Convert to \copy or aws_s3
CREATE SUBSCRIPTION	Replication config	Must use RDS-specific	Adjust connection parameters

```
# Automated cleanup script for RDS/Aurora compatibility:
sed -i 's/TABLESPACE [a-zA-Z_]*/g' /apps/backups/mydb_ddl.sql
sed -i '/^ALTER .* OWNER TO/d' /apps/backups/mydb_ddl.sql
sed -i '/^GRANT /d' /apps/backups/mydb_ddl.sql
sed -i '/^REVOKE /d' /apps/backups/mydb_ddl.sql
# ALWAYS review manually after automated cleanup!
```

Step 3: Apply DDL to Target

```
# For RDS/Aurora:
psql -h mydb.xxxx.us-east-1.rds.amazonaws.com \
  -U masteruser -d mydb \
  -f /apps/backups/mydb_ddl.sql \
  --set ON_ERROR_STOP=on \
  2>&1 | tee /apps/logs/ddl_apply_$(date +%Y%m%d).log

# Review errors:
grep -i error /apps/logs/ddl_apply_$(date +%Y%m%d).log

# For EC2 self-managed:
psql -h localhost -U postgres -d mydb \
  -f /apps/backups/mydb_ddl.sql \
  --set ON_ERROR_STOP=on \
  2>&1 | tee /apps/logs/ddl_apply_$(date +%Y%m%d).log
```

Step 4: Validate DDL on Target

```
-- Compare object counts between source and target:
-- Run on BOTH source and target, compare results

SELECT 'Tables' AS obj, count(*) FROM pg_tables WHERE schemaname NOT IN ('pg_catalog','information_schema')
UNION ALL SELECT 'Indexes', count(*) FROM pg_indexes WHERE schemaname NOT IN ('pg_catalog','information_schema')
UNION ALL SELECT 'Views', count(*) FROM pg_views WHERE schemaname NOT IN ('pg_catalog','information_schema')
UNION ALL SELECT 'Mat.Views', count(*) FROM pg_matviews
UNION ALL SELECT 'Functions', count(*) FROM pg_proc p JOIN pg_namespace n ON p.pronamespace=n.oid
  WHERE n.nspname NOT IN ('pg_catalog','information_schema')
UNION ALL SELECT 'Triggers', count(*) FROM pg_trigger WHERE NOT tgisinternal
UNION ALL SELECT 'Sequences', count(*) FROM pg_sequences WHERE schemaname NOT IN ('pg_catalog','information_schema')
UNION ALL SELECT 'Constraints', count(*) FROM pg_constraint c JOIN pg_namespace n ON c.connamespace=n.oid
  WHERE n.nspname NOT IN ('pg_catalog','information_schema')
ORDER BY 1;
```

6. Section 4 — Migrating Data Separately

When DDL has been migrated separately (Section 5 above), data migration loads only the table data, sequences, and large objects into the pre-existing schema.

Method A: `pg_dump --data-only` + `pg_restore`

```
# Export data only (custom format for parallel restore)
pg_dump -h localhost -U postgres -d mydb \
  --data-only \
  --format=custom \
  --compress=6 \
  --jobs=4 \
  --file=/apps/backups/mydb_data_$(date +%Y%m%d).dump

# Restore data only to target (DDL must already exist)
pg_restore -h mydb.xxxx.us-east-1.rds.amazonaws.com \
  -U masteruser -d mydb \
  --data-only \
  --jobs=4 \
  --disable-triggers \
  --no-owner \
  --verbose \
  /apps/backups/mydb_data_$(date +%Y%m%d).dump \
  2>&1 | tee /apps/logs/data_restore_$(date +%Y%m%d).log
```

CRITICAL — Disable Triggers During Data Load

Use `--disable-triggers` when loading data separately. Without this, triggers fire during INSERT operations, causing duplicate side effects, constraint violations, and massive performance degradation. On RDS/Aurora, the restore user must be a member of `rds_superuser` to disable triggers.

Method B: COPY-based bulk export/import (per-table)

```
# Export each table to CSV using COPY (on source):
psql -h localhost -U postgres -d mydb -c \
  "\COPY public.large_table TO '/apps/backups/large_table.csv' WITH (FORMAT csv, HEADER true)"

# Import on target:
psql -h mydb.xxxx.us-east-1.rds.amazonaws.com -U masteruser -d mydb -c \
  "\COPY public.large_table FROM '/apps/backups/large_table.csv' WITH (FORMAT csv, HEADER true)"

# For RDS: Use aws_s3 extension for server-side import:
-- On RDS target:
CREATE EXTENSION IF NOT EXISTS aws_s3;
SELECT aws_s3.table_import_from_s3(
  'public.large_table',
  '',
  '(FORMAT csv, HEADER true)',
  aws_commons.create_s3_uri('my-migration-bucket', 'data/large_table.csv', 'us-east-1')
);
```

Method C: AWS DMS (Data Only — Full Load)

Create a DMS task with migration type **'Migrate existing data only'** when DDL already exists on target. Set table preparation mode to **'Do nothing'** (preserves existing schema). This is the fastest method for large datasets that need minimal downtime.

Sequence Synchronization (Required for All Methods):

```
-- Generate sequence sync commands on SOURCE:
SELECT 'SELECT setval('' || schemaname || '.' || sequencename || '', ' || last_value || ' )
;'
FROM pg_sequences
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
AND last_value IS NOT NULL;

-- Run the generated commands on TARGET after data load
-- Example output:
-- SELECT setval('public.users_id_seq', 458723);
-- SELECT setval('public.orders_id_seq', 1293847);
```


7. Section 5 — Combined DDL + Data Migration (Recommended Procedure)

This section provides the complete, end-to-end procedure for migrating everything in a single operation. This is the simplest approach for databases under 500 GB when downtime is acceptable.

Recommended Method for Combined Migration

Use **pg_dump (custom format) + pg_restore** with parallel jobs. This is the gold standard for PostgreSQL migrations and is recommended by AWS, EnterpriseDB, Crunchy Data, and the PostgreSQL community.

Complete Step-by-Step Procedure:

Step 1: Pre-Migration Backup (Safety Net)

```
# Take a full backup of source BEFORE any migration work
pg_basebackup -h localhost -U postgres \
  -D /apps/backups/pre_migration_$(date +%Y%m%d) \
  --wal-method=stream --checkpoint=fast --progress

# AND/OR pgBackRest backup:
pgbackrest --stanza=mydb-cluster --type=full backup
```

Step 2: Stop Application Traffic

```
# Option A: Revoke connections
ALTER DATABASE mydb SET allow_connections = false;
SELECT pg_terminate_backend(pid) FROM pg_stat_activity
  WHERE datname = 'mydb' AND pid != pg_backend_pid();

# Option B: Block at pg_hba.conf level
# Comment out application host entries, reload
```

Step 3: Final Checkpoint and Stats

```
CHECKPOINT;
ANALYZE;

-- Record final state for validation:
SELECT schemaname, tablename, n_live_tup
FROM pg_stat_user_tables ORDER BY n_live_tup DESC;
```

Step 4: Execute pg_dump

```
pg_dump -h localhost -U postgres -d mydb \
  --format=custom \
  --compress=6 \
  --jobs=4 \
  --verbose \
  --file=/apps/backups/mydb_full_$(date +%Y%m%d_%H%M%S).dump \
  2>&1 | tee /apps/logs/pg_dump_$(date +%Y%m%d).log

# Verify dump file integrity:
pg_restore --list /apps/backups/mydb_full_*.dump | head -20
ls -lh /apps/backups/mydb_full_*.dump
```

Step 5: Transfer to AWS

```
# Direct transfer via SCP (over VPN/Direct Connect):
scp -C /apps/backups/mydb_full_*.dump \
  ec2-user@<EC2_PRIVATE_IP>:/apps/backups/

# Or via S3:
aws s3 cp /apps/backups/mydb_full_*.dump \
  s3://migration-bucket/pg/ --sse AES256

# Verify transfer integrity:
md5sum /apps/backups/mydb_full_*.dump # Source
ssh ec2-user@<EC2_IP> 'md5sum /apps/backups/mydb_full_*.dump' # Target
```

Step 6: Create Target Database

```
# On target (RDS/Aurora/EC2):
psql -h <TARGET_ENDPOINT> -U masteruser -d postgres
CREATE DATABASE mydb;

# Install required extensions BEFORE restore:
\c mydb
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS uuid-oss;
-- Add all extensions from source assessment
```

Step 7: Execute pg_restore

```
pg_restore -h <TARGET_ENDPOINT> \
  -U masteruser -d mydb \
  --format=custom \
  --jobs=4 \
  --no-owner \
  --no-privileges \
  --verbose \
  /apps/backups/mydb_full_*.dump \
  2>&1 | tee /apps/logs/pg_restore_$(date +%Y%m%d).log

# Check for errors (non-zero exit = errors):
echo "Exit code: $?"
grep -c 'ERROR' /apps/logs/pg_restore_$(date +%Y%m%d).log
```

Step 8: Post-Restore Optimization

```
# Run ANALYZE to update statistics on target:
psql -h <TARGET_ENDPOINT> -U masteruser -d mydb \
  -c 'ANALYZE VERBOSE;'

# Rebuild any invalid indexes:
SELECT indexrelid::regclass, indisvalid FROM pg_index WHERE NOT indisvalid;
-- REINDEX INDEX CONCURRENTLY <invalid_index_name>;
```

8. Section 6 — Reverse Migration: AWS → Standalone VM

Reverse migration (repatriation from AWS back to on-premise/private cloud VMs) follows the same general methodology but with key differences in how you extract data from managed services.

From RDS/Aurora → Standalone VM:

[illegible]

From EC2 Self-Managed → Standalone VM:

This is the simplest reverse migration scenario since you have full OS and filesystem access on both sides. All six methods are available: `pg_dump/pg_restore`, `pg_basebackup`, `pgBackRest`, DMS, logical replication, and even `rsync` of the data directory (for same-version, same-platform migrations).

pgBackRest Reverse Migration Pattern

If your EC2 instance uses pgBackRest with an S3 repository, configure the on-premise VM's pgBackRest to point to the same S3 bucket and run **pgbackrest --stanza=mydb-cluster restore**. This is the fastest reverse migration path for large databases — the backup is already in S3.

9. Section 7 — Post-Migration Validation & Checks

Post-migration validation is mandatory. No migration is considered complete until all validation checks pass. This section provides a comprehensive validation framework used by enterprise migration teams.

Check 1: Object Count Comparison

```
-- Run on BOTH source and target, compare output:
SELECT 'Tables' AS obj_type, count(*) AS cnt FROM pg_tables
  WHERE schemaname NOT IN ('pg_catalog','information_schema')
UNION ALL SELECT 'Indexes', count(*) FROM pg_indexes
  WHERE schemaname NOT IN ('pg_catalog','information_schema')
UNION ALL SELECT 'Views', count(*) FROM pg_views
  WHERE schemaname NOT IN ('pg_catalog','information_schema')
UNION ALL SELECT 'Functions', count(*) FROM pg_proc p
  JOIN pg_namespace n ON p.pronamespace=n.oid
  WHERE n.nspname NOT IN ('pg_catalog','information_schema')
UNION ALL SELECT 'Triggers', count(*) FROM pg_trigger WHERE NOT tgisinternal
UNION ALL SELECT 'Sequences', count(*) FROM pg_sequences
  WHERE schemaname NOT IN ('pg_catalog','information_schema')
ORDER BY 1;
```

Check 2: Row Count Comparison (Every Table)

```
-- Generate row count queries for all user tables:
SELECT 'SELECT ''' || schemaname || '.' || tablename || ''' AS tbl, count(*) AS cnt FROM '
  || schemaname || '.' || tablename || ' UNION ALL'
FROM pg_tables WHERE schemaname NOT IN ('pg_catalog','information_schema')
ORDER BY schemaname, tablename;

-- Run output (remove trailing UNION ALL) on BOTH source and target
-- Compare results – counts MUST match exactly
```

Check 3: Data Integrity — Checksum Validation

```
-- For critical tables, compute MD5 of sorted data:
-- Run on BOTH source and target:
SELECT md5(string_agg(t::text, '')) FROM (
  SELECT * FROM public.important_table ORDER BY primary_key_col
) t;

-- For large tables, sample-based validation:
SELECT count(*), sum(hashtext(t::text)) FROM (
  SELECT * FROM public.large_table TABLESAMPLE SYSTEM(1) -- 1% sample
) t;
```

Check 4: Sequence Values

```
-- Verify sequences are synced (especially after DMS or logical replication):
SELECT schemaname, sequencename, last_value, start_value, increment_by
FROM pg_sequences
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY schemaname, sequencename;

-- Compare output between source and target
-- last_value on target must be >= source value
```

Check 5: Index Validity

```
-- Check for invalid indexes on target:
SELECT indexrelid::regclass AS index_name, indrelid::regclass AS table_name, indisvalid
FROM pg_index WHERE NOT indisvalid;

-- If invalid indexes found, rebuild:
-- REINDEX INDEX CONCURRENTLY <index_name>;
```

Check 6: Constraint Verification

```
-- Check constraint status on target:
SELECT conname, conrelid::regclass, contype, convalidated
FROM pg_constraint
WHERE NOT convalidated;

-- Validate any not-validated constraints:
-- ALTER TABLE <table> VALIDATE CONSTRAINT <constraint_name>;
```

Check 7: Extension Verification

```
-- Compare extensions:
SELECT extname, extversion FROM pg_extension ORDER BY extname;
```

Check 8: Performance Baseline

```
-- Run ANALYZE on all tables (update planner statistics):
ANALYZE VERBOSE;

-- Run representative queries and compare execution plans:
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) SELECT ...; -- Your critical queries

-- Check for missing indexes or plan regressions:
SELECT schemaname, relname, seq_scan, idx_scan, n_live_tup
FROM pg_stat_user_tables
WHERE seq_scan > 100 AND idx_scan = 0 AND n_live_tup > 10000
ORDER BY seq_scan DESC;
```

Check 9: Application Connectivity Test

- Update application connection strings to point to new target
- Test all CRUD operations from application layer
- Verify SSL/TLS connectivity from application
- Run application smoke tests / regression suite
- Monitor error logs on both application and database sides

Check 10: Monitoring & Alerting Setup

- Verify CloudWatch metrics are flowing (RDS/Aurora)
- Confirm Performance Insights is capturing query data
- Set up pgBackRest scheduled backups on EC2 targets
- Configure log shipping / WAL archiving
- Test point-in-time recovery capability

10. Section 8 — Enterprise Best Practices & MNC Patterns

This section documents the migration patterns and best practices used by top-tier technology companies, system integrators, and AWS Professional Services in production migrations.

10.1 AWS Migration Acceleration Program (MAP) Framework

The MAP framework is the standard used by AWS Professional Services and partner SIs (Accenture, Deloitte, Wipro, Infosys, TCS) for enterprise database migrations:

- **Phase 1 — Assess (2–4 weeks):** Inventory all databases, run AWS SCT assessment reports, identify blockers, estimate effort and costs, build migration wave plan
- **Phase 2 — Mobilize (2–4 weeks):** Set up AWS landing zone, configure networking (Direct Connect/VPN), build CI/CD for database deployments, conduct proof-of-concept migration
- **Phase 3 — Migrate (per wave):** Execute migration runbooks (this document), validate, cutover
- **Phase 4 — Optimize (ongoing):** Right-size instances, tune parameters, implement monitoring, enable Reserved Instances

10.2 Migration Wave Planning (MNC Standard)

Enterprise migrations NEVER migrate all databases at once. They use a wave-based approach:

Wave	Databases	Purpose	Timeline
Wave 0 (Pilot)	1–2 non-critical DBs	Validate tooling, process, networking	Week 1–2
Wave 1	Dev/Test environments	Build confidence, train team	Week 3–4
Wave 2	Low-risk production DBs	First production validation	Week 5–6
Wave 3	Medium-risk production DBs	Scale migration process	Week 7–10
Wave 4	High-risk / large production DBs	Critical workloads	Week 11–14
Wave 5 (Final)	Remaining databases	Complete migration	Week 15+

10.3 Key Best Practices from MNC Migrations

Dry Run Everything: Never execute a migration for the first time in production. Run at least 2 full dry runs in a staging environment identical to production. Measure duration, identify issues, and refine the runbook. Top MNCs run 3–5 rehearsals for critical databases.

Parallel Validation: Run the source and target databases in parallel for 24–72 hours post-migration before decommissioning the source. Route read traffic to both and compare results.

Automated Rollback: Every migration plan MUST include a tested rollback procedure. If cutover fails, you must be able to revert to the source within the maintenance window.

Change Freeze: Implement a change freeze on the source database 48–72 hours before migration. No DDL changes, no application deployments, no parameter changes.

Communication Plan: Send migration notifications to all stakeholders: 72 hours before, 24 hours before, at migration start, at completion, and at sign-off.

Checksum-Based Validation: Do NOT rely solely on row counts. Use checksums or hash-based validation for critical tables. `pg_dump` with `--data-only` can be used to create a validation dump on the target for comparison.

Sequence Safety Margin: When syncing sequences, add a safety margin: set target sequence value to source value + 1000. This prevents collision if any late-arriving transactions are in flight.

DNS-Based Cutover: Use DNS CNAME records for database endpoints. Cutover becomes a DNS update rather than application configuration changes. Lower TTL to 60 seconds before migration window.

Post-Migration Monitoring (Soak Period): Monitor the target database intensively for 72 hours post-migration: query latency, error rates, connection counts, replication lag (if applicable). Do not decommission the source until the soak period passes.

Documentation: Document everything: actual migration duration, any issues encountered, deviations from runbook, and lessons learned. Feed these back into the runbook for future migrations.

11. Section 9 — Rollback Procedures

Rollback is NOT Optional

Every production migration **MUST** have a tested rollback plan. If the migration or cutover fails, you must be able to restore service on the original source within the maintenance window. Rollback procedure should be rehearsed during dry runs.

Rollback Scenario 1: Cutover Not Yet Performed

If validation fails BEFORE application cutover, simply discard the target and re-enable the source:

```
-- Re-enable connections on source:
ALTER DATABASE mydb SET allow_connections = true;

-- Verify source is functioning:
SELECT count(*) FROM pg_stat_activity WHERE datname = 'mydb';

-- Drop or clean up target database if needed
```

Rollback Scenario 2: Cutover Performed, Issues Detected

If application has been pointed to the new target but issues are detected within the soak period:

```
# 1. Stop application traffic to AWS target
# 2. If using DMS with CDC, the source still has all data
#    → Point application back to source
# 3. If using pg_dump (offline), restore from pre-migration backup:
pg_restore -h localhost -U postgres -d mydb \
  --clean --create \
  /apps/backups/pre_migration_backup.dump

# 4. If using pgBackRest:
pgbackrest --stanza=mydb-cluster --delta restore

# 5. Update DNS/connection strings back to source
# 6. Verify application connectivity to source
```

12. Section 10 — Troubleshooting Common Issues

Issue	Symptom	Root Cause	Resolution
pg_restore error: role does not exist	ERROR: role 'appuser' does not exist	Source had roles not created on target	Use --no-owner --no-privileges, or pre-create roles
pg_restore error: extension not available	ERROR: could not open extension control file	Extension not supported on RDS	Remove from DDL or find RDS-compatible alternative
DMS task fails: WAL segment removed	WAL segment has already been removed	Source recycles WAL before DMS reads it	Increase wal_keep_size or use replication slot
DMS CDC latency increasing	CDCLatencySource keeps growing	Source write volume exceeds DMS throughput	Scale up DMS instance; reduce source write rate during migration
Logical replication: relation does not exist	ERROR: relation 'public.newtable' does not exist	Table created after publication but not on subscriber	Re-create DDL on target; refresh subscription
Slow pg_restore	Restore taking 3× longer than expected	Not using parallel jobs or network bottleneck	Use --jobs=4-8; restore from EC2 in same VPC/AZ
Sequence values not synced	Duplicate key errors post-migration	DMS / logical rep don't sync sequences	Run sequence sync script (see Section 6)
SSL connection required error	FATAL: no pg_hba.conf entry ... SSL off	RDS forces SSL; client not using SSL	Add sslmode=require to connection string
Permission denied on RDS	ERROR: permission denied for schema/table	Need rds_superuser role	GRANT rds_superuser TO migration_user;
pg_dump: too many clients	FATAL: too many connections	Parallel dump exceeds max_connections	Increase max_connections or reduce --jobs

13. Appendix A — Command Quick Reference

pg_dump Commands:

```
# Full dump (custom format, compressed, parallel)
pg_dump -Fc -Z6 -j4 -f /apps/backups/full.dump mydb

# Schema only
pg_dump --schema-only -f /apps/backups/ddl.sql mydb

# Data only
pg_dump --data-only -Fc -j4 -f /apps/backups/data.dump mydb

# Single schema
pg_dump --schema=myschema -Fc -f /apps/backups/schema.dump mydb

# Single table
pg_dump --table=public.mytable -Fc -f /apps/backups/table.dump mydb

# Exclude table(s)
pg_dump --exclude-table=public.audit_log -Fc -f /apps/backups/no_audit.dump mydb
```

pg_restore Commands:

```
# Full restore (parallel, no owner, no privileges)
pg_restore -j4 --no-owner --no-privileges -d mydb /apps/backups/full.dump

# Data only (disable triggers)
pg_restore --data-only --disable-triggers -d mydb /apps/backups/data.dump

# Schema only
pg_restore --schema-only -d mydb /apps/backups/full.dump

# List contents of dump file
pg_restore --list /apps/backups/full.dump

# Restore specific table from dump
pg_restore -d mydb --table=mytable /apps/backups/full.dump
```

pgBackRest Commands:

```
# Full backup
pgbackrest --stanza=mydb-cluster --type=full backup

# Differential backup
pgbackrest --stanza=mydb-cluster --type=diff backup

# Restore (with delta)
pgbackrest --stanza=mydb-cluster --delta restore

# Info (list backups)
pgbackrest --stanza=mydb-cluster info

# Verify backups
pgbackrest --stanza=mydb-cluster verify
```

AWS CLI Commands:

```
# Upload dump to S3
aws s3 cp /apps/backups/full.dump s3://bucket/pg/ --sse AES256

# Download from S3
aws s3 cp s3://bucket/pg/full.dump /apps/backups/

# Create RDS snapshot
aws rds create-db-snapshot --db-instance-identifier mydb --db-snapshot-identifier pre-migration

# Describe DMS tasks
aws dms describe-replication-tasks --query 'ReplicationTasks[*].{Name:ReplicationTaskIdentifier,Status:Status}'
```

14. Appendix B — Migration Checklist (Printable)

Pre-Migration (1-2 Weeks Before)

- Source database assessment completed (sizes, versions, extensions, object counts)
- AWS target provisioned and configured (RDS/Aurora/EC2, Parameter Groups, Security Groups)
- Network connectivity verified (VPN/Direct Connect, firewall rules, port 5432 open)
- Migration user created with appropriate privileges
- SSL/TLS configured on both source and target
- Dry run #1 completed in staging environment
- Dry run #2 completed — duration and issues documented
- Rollback procedure documented and tested
- Maintenance window scheduled and communicated to stakeholders
- Change freeze enacted on source (48–72 hours before)
- DNS TTL lowered to 60 seconds (if using DNS-based cutover)
- Monitoring and alerting configured on target

Migration Day — Pre-Execution

- Full backup of source taken (pg_basebackup AND/OR pgBackRest)
- Backup verified (restore test or pgbackrest verify)
- Application traffic stopped / connections blocked
- Final CHECKPOINT and ANALYZE on source
- Source row counts and checksums recorded
- All stakeholders notified: migration started

Migration Day — Execution

- pg_dump / DMS / pgBackRest backup executed successfully
- Dump file transferred to AWS (checksum verified)
- pg_restore / DMS full load completed
- Restore log reviewed — errors addressed
- ANALYZE VERBOSE run on target

Post-Migration Validation

- Object count comparison: source vs target (match)
- Row count comparison: all tables (match)
- Checksum validation on critical tables (match)
- Sequence values synced (target $\geq$ source + margin)
- Invalid indexes: none found
- Constraints validated
- Extensions verified
- Application connectivity test passed

Application smoke test / regression suite passed

Query performance baseline acceptable

Cutover & Soak

DNS / connection strings updated to target

Application functional with new target

Monitoring active: CloudWatch, Performance Insights, pg logs

72-hour soak period started

No critical issues during soak → Migration SIGNED OFF

Source database access restricted (read-only or stopped)

Migration credentials rotated / removed

Documentation updated: lessons learned, actual timelines

End of Document — PostgreSQL Migration Runbook v2.0 — April 2026